Guest
:RegisterView
:AuthenticationRoute
:AuthenticationController
:AuthenticationService
:AuthenticationRepository
:PrismaService
Database
1. Enter email, click "Send OTP"
2. POST /api/auth/otp
3. Send request by post method
4. sendOTP(dto)
5. findUserByEmail(email)
6. user.findUnique({ where: { email, deletedAt: null } })
7. Execute query
8. Return result
9. Return result
10. Return result
11. Handle check email existence
alt
[Email already exists]
11.1. Return error 422 EMAIL_ALREADY_EXISTS
11.1.1. Return error
11.1.2. Return error
11.1.3. Display error message
[Email does not exist]
12. findVerificationCodeByEmailAndType(email, type)
13. verificationCode.findUnique({ email_type })
14. Execute query
15. Return result
16. Return result
17. Return result
18. Handle check OTP attempts
alt
[Too many attempts (attempts > 5)]
18.1. Return error 422 TOO_MANY_ATTEMPTS
18.1.1. Return error
18.1.2. Return error
18.1.3. Display error message
[Attempts <= 5]
19. generateOTP(), set expiresAt, attempts++
20. upsertVerificationCode(data)
21. verificationCode.upsert(...)
22. Execute upsert
23. Return result
24. Return result
25. Return result
26. Handle save OTP result
alt
[Save OTP not success]
26.1. Return error
26.1.1. Return error
26.1.2. Return error
26.1.3. Display error message
[Save OTP success]
26.2. Send OTP email via Gmail SMTP
26.2.1. Return success
26.2.2. Return success
26.2.3. Return success
26.2.4. Display OTP sent message
27. Fill in form, click "Register"
28. POST /api/auth/register
29. Send request by post method
30. register(dto)
31. findUserByEmail(email)
32. user.findUnique({ where: { email, deletedAt: null } })
33. Execute query
34. Return result
35. Return result
36. Return result
37. Handle user validation result
alt
[User is banned]
37.1. Return error 403 USER_BANNED
37.1.1. Return error
37.1.2. Return error
37.1.3. Display error message
[Email already exists]
37.2. Return error 422 EMAIL_ALREADY_EXISTS
37.2.1. Return error
37.2.2. Return error
37.2.3. Display error message
[Email does not exist]
38. validateVerificationCode(email, code, type)
39. verificationCode.findFirst({ code, email, type })
40. Execute query
41. Return result
42. Return result
43. Return result
44. Handle OTP verification result
alt
[OTP invalid or expired]
44.1. Return error 422 INVALID_VERIFICATION_CODE / OTP_EXPIRED
44.1.1. Return error
44.1.2. Return error
44.1.3. Display error message
[OTP valid]
45. getRoleId(role)
46. role.findFirst({ name, deletedAt: null })
47. Execute query
48. Return result
49. Return result
50. Return result
51. Handle role lookup result
alt
[Role not found]
51.1. Return error
51.1.1. Return error
51.1.2. Return error
51.1.3. Display error message
[Role found]
52. hash(password) - bcrypt
53. createUserAndRegister(otp, user)
54. $transaction([deleteMany, user.create])
55. Execute transaction
56. Return result
57. Return result
58. Return result
59. Handle user creation result
alt
[Create user not success]
59.1. Return error
59.1.1. Return error
59.1.2. Return error
59.1.3. Display error message
[Create user success]
59.2. Return success
59.2.1. Return success
59.2.2. Return success
59.2.3. Display success, go to /login